

ExoDetect:

From Tiny Light Dips to Possible New Worlds

Can machine learning help us decide which Kepler candidates deserve a second look?

What if one tiny shadow is a new world?!

- Kepler was a space telescope that searched for exoplanets.
- It did this by watching the brightness of stars.
- When a planet passes in front of its star, the light becomes a little bit weaker.
- This tiny drop in brightness can be a clue that a planet exists.
- Not every light dip is a real planet; some signals are false positives.
- Machine learning can help rank the most promising candidates for further study.





Kepler Dataset & ML Setup

What data was used, what features were selected, and what EDA was done?



1 Dataset Snapshot



				
Total KOIs	Confirmed	False Positives	Candidates	Labeled training pool used for classification
9,564	2,747	4,839	1,978	7,586

2 ML Goal in This Project



- Train a classifier on known labels: CONFIRMED vs FALSE POSITIVE
- Predict planet-like probabilities for unresolved CANDIDATES
- Rank candidates for follow-up priority

3 Selected Features

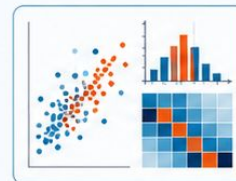


Feature	Meaning
koi_period	orbital period
koi_prad	planet radius
koi_duration	transit duration
koi_depth	transit depth
koi_model_snr	signal-to-noise ratio
koi_impact	transit geometry / impact parameter
koi_teq	equilibrium temperature
koi_insol	incident stellar flux
koi_steff	stellar effective temperature
koi_slogg	stellar surface gravity
koi_srad	stellar radius
koi_kepmag	Kepler magnitude

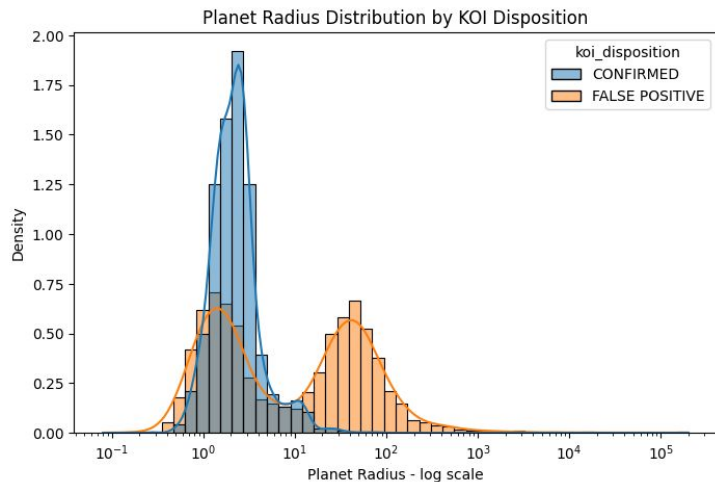
4 EDA Performed



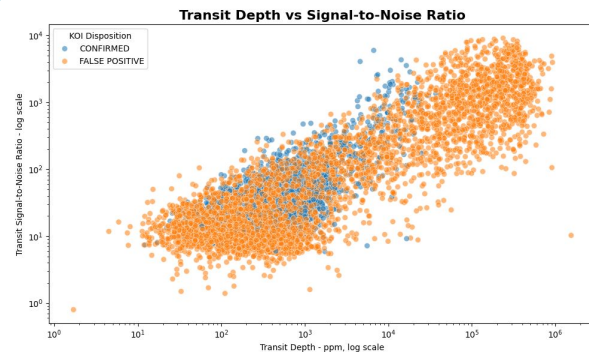
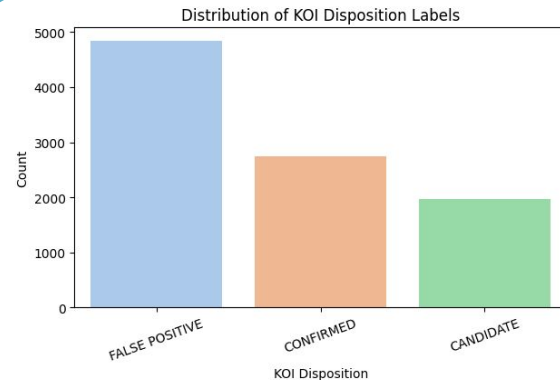
- Class distribution of KOI labels
- Missing values in selected features
- Radius vs orbital period
- Planet radius distribution by disposition
- Signal-to-noise ratio vs radius
- Correlation heatmap of selected numerical features



Can the data give us scientific clues before ML ?



- Radius is a useful clue: false positives often show larger or more extreme radius values.
- The dataset is imbalanced, with more false positives than confirmed planets or candidates.
- Transit depth and signal strength show patterns, but the classes still mix, so ML is needed.



Which ML model performed best?

Classification task: CONFIRMED vs FALSE POSITIVE



Training pipeline

What happened before candidate ranking?

Selected features radius, period, depth, SNR...

Train / test split test data was unseen

Fit several models LR, SVM, GB, RF

Compare scores Macro F1 + ROC-AUC

Select final model Tuned Random Forest



Model comparison on test set

Model	Accuracy	F1 Macro	ROC-AUC
Random Forest	0.934	0.929	0.978
Gradient Boosting	0.926	0.920	0.976
SVM	0.847	0.842	0.931
Logistic Regression	0.828	0.823	0.903

Selected model
Tuned Random Forest

Why these metrics?
Macro F1 handles imbalance
ROC-AUC supports probabilities

Best model selected for the next step: predicting planet-like probabilities for unresolved CANDIDATES.



How did classification become candidate ranking?

The ML model first learns a classification task, then its probabilities are used to rank unresolved Kepler candidates.

From training to ranking

Important distinction: CANDIDATES were not used to train the classifier.

1

Train on known labels

CONFIRMED = 1 vs FALSE POSITIVE = 0

ML training

2

Check on unseen test data

The test set has real labels, so scores can be measured.

Evaluation

3

Apply to unresolved CANDIDATES

These objects have no confirmed answer in the dataset.

Prediction use

4

Create planet_probability

Probability of class 1: $P(\text{CONFIRMED} \mid \text{selected features})$.

ML output

5

Sort candidates by probability

Highest probability = most planet-like shortlist item.

Ranking

Exact step in the notebook

```
X_candidates = candidate_df[feature_cols]
planet_probability = best_pipeline.predict_proba(X_candidates)[:, 1]
rank = sort by planet_probability ↓
```

[:, 1] means the probability of the positive class: CONFIRMED.

Top candidate ranking preview

Candidate	Planet-like probability	Radius	SNR
K01145.01	0.993	2.32	33.1
K00426.01	0.987	3.84	62.4
K01972.01	0.987	2.88	39.8
K01861.01	0.987	3.56	47.5
K04526.01	0.983	2.72	15.0

Final output: Not official planet confirmation , a smart shortlist for follow-up study.

Which unresolved candidates look most planet-like?

The trained model ranks unresolved Kepler candidates by predicted planet-like probability.



Why is it interesting?

- 1 High model probability
- 2 Reasonable planet radius
- 3 Strong transit signal / Good follow-up target

★ Top candidate ranking preview

Candidate	Probability	Radius	SNR	Priority
KOI-K01145.01	99.3	2.32	33.1	High Priority
KOI-K00426.01	98.7	3.84	62.4	High Priority
KOI-K01972.01	98.7	2.88	39.8	High Priority
KOI-K01861.01	98.7	3.56	47.5	High Priority
KOI-K04526.01	98.3	2.72	15.0	High Priority

Probability = model confidence that a candidate is planet-like (class 1: CONFIRMED).

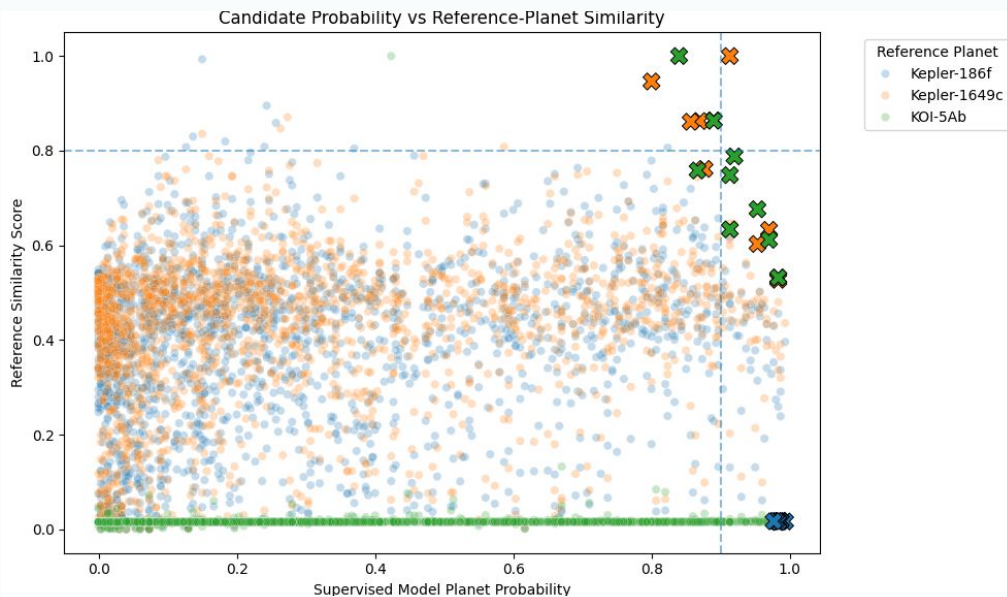
◆ Top candidate spotlight

- 🎯 Candidate: **KOI-K01145.01**
- ✓ Planet-like probability: **99.3**
- 📏 Radius: **2.32**

Final output: A prioritized shortlist for follow-up observation.

Reference similarity: a smarter shortlist

High model probability + high similarity to known planets = stronger follow-up targets.



What this adds

Score = 70% probability + 30% similarity

- 1 Model probability stays the main signal.
- 2 Similarity highlights candidates close to known planets.
- 3 Upper-right points become follow-up targets, not official discoveries.

Example: KO2159.01 ranked high and is still a candidate → useful for future follow-up.

Takeaway: ExoDetect does not confirm planets — it prioritizes promising KOIs for deeper checks with light curves, TESS data, and expert follow-up.

Can AI discover planets?

Not alone — but it can help us decide where to look first.



What worked

- Built a full ML pipeline
- Compared several models
- Ranked unresolved candidates



Limitations

- Not official planet confirmation
- No raw light curve analysis
- Based on tabular Kepler features



Future work

- Use raw light curves
- Validate with TESS data
- Add explainability with SHAP

Final answer: ML turns thousands of signals into a smarter shortlist for follow-up study.

Thank You!

Questions?



Thank you for your time, support, and attention.

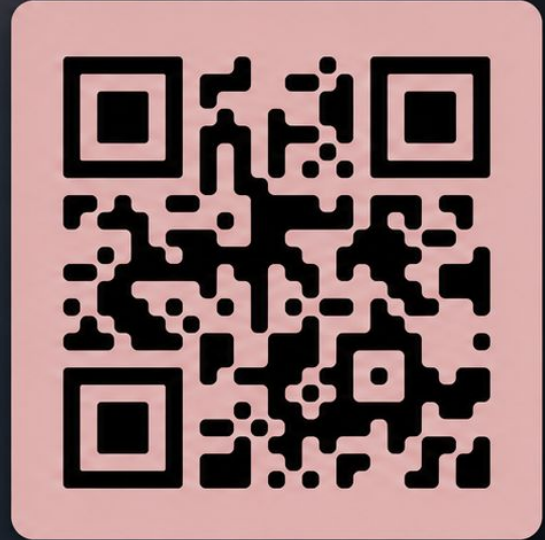


Let's connect on LinkedIn — scan the QR code.



Dalileh Oladzadeh

EXOPLANETS • MACHINE LEARNING • DISCOVERY



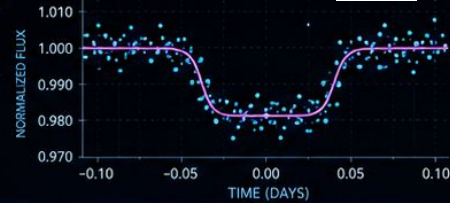
Scan to connect on LinkedIn

Evaluation Metrics Summary

For Exoplanet Classification Models



KEPLER LIGHT CURVE (EXAMPLE)



1



Accuracy

Overall proportion of correct predictions

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

2



Precision

Of predicted positives, how many are truly positive?

$$\text{Precision} = \frac{TP}{TP + FP}$$

3



Recall

Of actual positives, how many did the model find?

$$\text{Recall} = \frac{TP}{TP + FN}$$

4



F1 Score

Balanced measure of Precision and Recall

$$\begin{aligned} F1 &= \frac{2 \times (\text{Precision} \times \text{Recall})}{\text{Precision} + \text{Recall}} \\ &= \frac{2TP}{2TP + FP + FN} \end{aligned}$$

5



AUC-ROC

Measures separability across classification thresholds

ROC: plot of TPR vs FPR

$$\text{TPR} = \frac{TP}{TP + FN}$$

$$\text{FPR} = \frac{FP}{FP + TN}$$

AUC = Area Under the ROC Curve



TP = True Positive

TN = True Negative

FP = False Positive

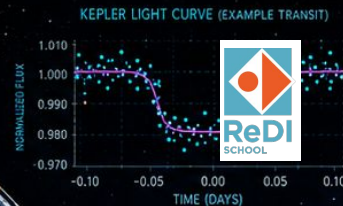
FN = False Negative

Models Used in My Project

• How each model works + a key formula •



Kepler Exoplanet Candidate Classification



1



Decision Tree

Splits the data step by step using the best feature threshold.

$$\text{Information Gain} = H(\text{parent}) - \sum (n_j / n) H(\text{child}_j)$$

★ Chooses the split with highest information gain.

2



Random Forest

Builds many decision trees and combines their votes.

$$\hat{y} = \text{majority_vote}(T_1(x), T_2(x), \dots, T_B(x))$$

★ Reduces overfitting and improves stability.

3



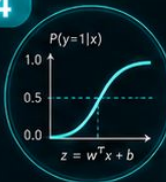
XGBoost

Adds trees sequentially to correct previous errors.

$$\hat{y}_i = \sum f_t(x_i)$$

★ Boosting model with strong performance on tabular data.

4



Logistic Regression

Estimates the probability of the positive class.

$$P(y=1|x) = 1 / (1 + e^{-(w^T x + b)})$$

★ Uses the sigmoid function for classification.

5



KNN

Looks at the nearest samples and predicts by neighbor vote.

$$\hat{y} = \text{mode}(\{y_i : x_i \in N_k(x)\})$$

★ Prediction depends on the k nearest neighbors.

6



SVM

Finds the boundary that maximizes the margin between classes.

$$f(x) = \text{sign}(w^T x + b)$$

★ Aims for the widest possible separating margin.

x = input features

$\hat{y}$ = predicted class

H = entropy

w, b = model parameters

